IT112 Web System and Technologies
Phase 6: Update and Delete Functionalities
Lab Report by SURVIVORS

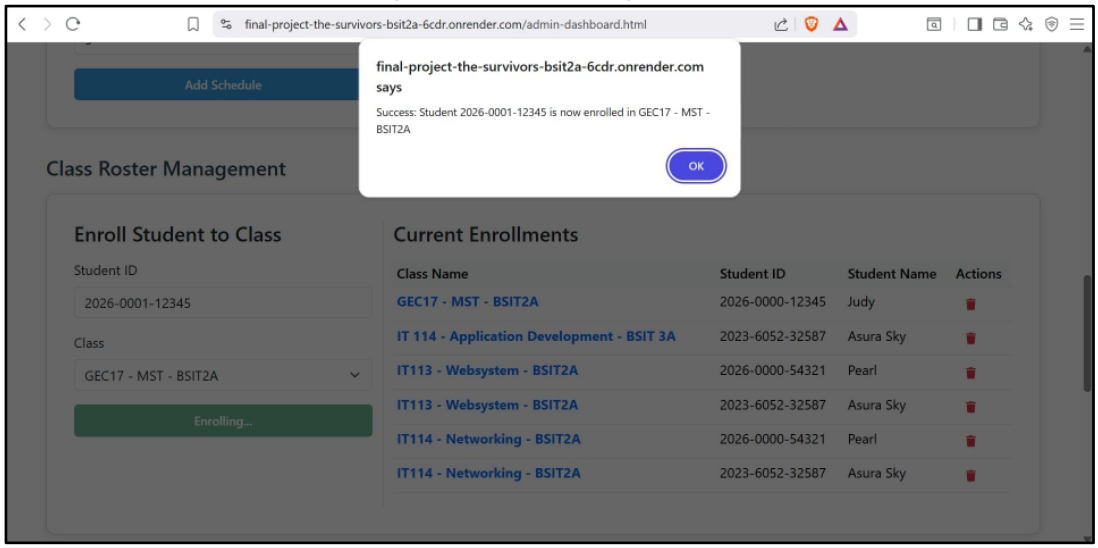
Group Members:

Jexson Sedon
John Roan Ballester
Michelle Diaz
Judy Pearl Balictar
Alyssa Jenille Reantaso
Ronel Garcia

Submitted to:

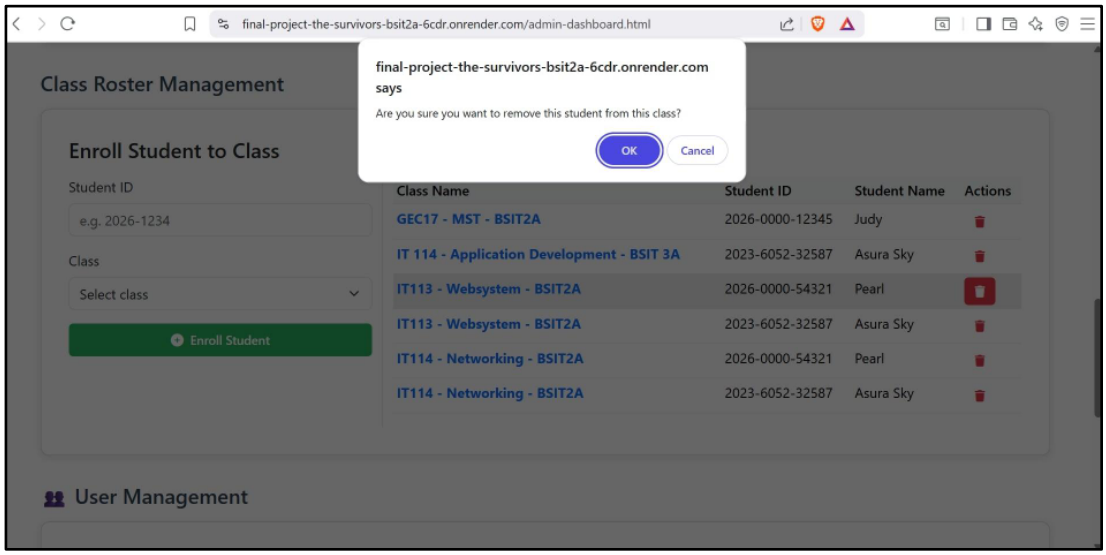
Guillermo V. Red,Jr,DIT
Assistant Professor IV

I. Screenshot of Working Edit Modal or Page



Notes:
This screenshot shows the edit modal or update page used to modify existing users. The form is automatically prefilled using data fetched from the backend API.

II. Screenshot of Delete Confirmation



Notes:
This screenshot shows the delete confirmation prompt displayed before removing a document from the database. This confirmation helps prevent accidental deletion of records.

III. Screenshot of MongoDB Database Updates

a. Edit

Notes:
This screenshot shows the successful editing of a user account in MongoDB. The account information for the user named “Benten” was updated and changed to “Justine” using the system’s edit functionality. The updated data was successfully reflected in the database, confirming that the update feature is functioning properly through the backend API and MongoDB integration.

b. Delete

Notes:
This screenshot shows the successful deletion of a user record from MongoDB. The account of the user named “Justine” was deleted from the database and replaced with a new account named “Sedon Jexson.” The changes confirm that the delete and add functionalities of the system are working properly through the backend API and MongoDB database.

IV. JavaScript Snippet for Edit Function

```
window.editRecord = async function(id) {
  const newStatus = prompt('Enter new status (Early, On-Time, Late, Absent):');
  if (!newStatus) return;

  try {
    const response = await fetch(`http://localhost:3000/api/attendance/${id}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      body: JSON.stringify({ status: newStatus })
    });

    const data = await response.json();
    if (response.ok) {
      alert('Record updated successfully');
      loadAttendance();
      loadStats();
    } else {
      alert(data.message || 'Failed to update record');
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Failed to update record');
  }
};
```

Notes:
This JavaScript code updates attendance records by sending a PUT request to the backend API with modified data. The function allows the admin to edit attendance status dynamically, and the updated information is immediately reflected on the dashboard after a successful update operation.

V. JavaScript Snippet for Delete Function

```
window.deleteUser = async function(id) {
  if (!confirm('Are you sure you want to delete this user?')) return;

  try {
    const response = await fetch(`http://localhost:3000/api/auth/users/${id}`, {
      method: 'DELETE',
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });

    const data = await response.json();
    if (response.ok) {
      alert('User deleted successfully');
      loadUsers();
    } else {
      alert(data.message || 'Failed to delete user');
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Failed to delete user');
  }
};
```

```
window.deleteRecord = async function(id) {
  if (!confirm('Are you sure you want to delete this record?')) return;

  try {
    const response = await fetch(`http://localhost:3000/api/attendance/${id}`, {
      method: 'DELETE',
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });

    const data = await response.json();
    if (response.ok) {
      alert('Record deleted successfully');
      loadAttendance();
      loadStats();
    } else {
      alert(data.message || 'Failed to delete record');
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Failed to delete record');
  }
};
```

Notes:
This JavaScript code handles the deletion of user and attendance records by sending a DELETE request to the backend API after user confirmation. Once the deletion is successful, the displayed data is automatically refreshed to reflect the updated database records.

VI. Backend Route and Controller Snippets

a. Backend route

```
router.get('/my-attendance', protect, authorize('student'), getMyAttendance);

router.get('/', protect, authorize('teacher', 'admin'), getAll);
router.get('/stats', protect, authorize('teacher', 'admin'), getStats);
router.get('/student-summary', protect, authorize('teacher', 'admin'), getStudentSummary);
router.get('/:id', protect, authorize('teacher', 'admin'), getById);
router.put('/:id', protect, authorize('admin'), update);
router.delete('/:id', protect, authorize('admin'), deleteAttendance);

module.exports = router;
```

```
const router = require('express').Router();
const { register, login, getAllUsers } = require('../controllers/authController');
const { protect, authorize } = require('../middleware/authMiddleware');
const { registerValidation, loginValidation, handleValidationErrors } = require('../middleware/valida
```

Notes:
These backend route definitions manage API endpoints for attendance and user operations using Express.js. The routes handle requests for retrieving, updating, and deleting records while also implementing authentication and authorization middleware to ensure secure access control.

b. Controller Snippet

- Edit

```
exports.update = async (req, res) => {
  try {
    const updated = await Attendance.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    ).lean();

    if (!updated) {
      return res.status(404).json({ success: false, message: 'Record not found' });
    }

    cache.del('attendance_all');
    cache.del(`attendance_${req.params.id}`);
    cache.del(`attendance_student_${updated.studentId}`);
    cache.del('attendance_student_summary');

    res.json({
      success: true,
      message: 'Updated successfully',
      data: updated
    });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};
```

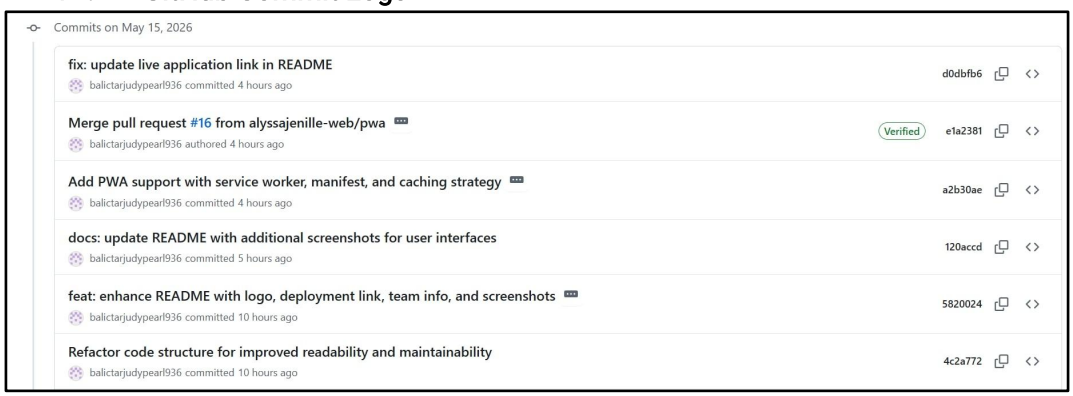
Notes:
This backend controller processes attendance record updates using MongoDB's findByIdAndUpdate() method. It validates the request, updates the selected document in the database, clears cached data, and returns the updated record as a response to the frontend.

- Delete

```
319 exports.deleteAttendance = async (req, res) => {
320   try {
321     const record = await Attendance.findByIdAndDelete(req.params.id);
322
323     if (!record) {
324       return res.status(404).json({ success: false, message: 'Record not found' });
325     }
326
327     cache.del('attendance_all');
328     cache.del(`attendance_${req.params.id}`);
329     cache.del(`attendance_student_${record.studentId}`);
330     cache.del('attendance_student_summary');
331
332     res.json({
333       success: true,
334       message: 'Deleted successfully'
335     });
336   } catch (error) {
337     res.status(500).json({ success: false, message: error.message });
338   }
339 };
```

Notes:
This backend controller handles record deletion using MongoDB's findByIdAndDelete() method. After successfully removing the selected attendance record from the database, the system clears related cached data and sends a success response back to the frontend application.

VII. GitHub Commit Logs



Notes:
This screenshot shows the GitHub commit history for Phase 6, including updates related to edit and delete functionalities, CRUD integration, and frontend/backend improvements. Similar to the previous phase, the GitHub Manager was still experiencing poor internet connection in their area and could not consistently access their laptop to manage repository tasks. Because of this, the Frontend Manager temporarily handled the GitHub responsibilities, including committing, merging, and synchronizing the project updates to ensure the group completed the required tasks successfully.

VIII. Reflections of Each Member

Project Manager (Jexson Sedon)

This phase helped me improve my leadership and coordination skills while managing the development of the dashboard and data rendering features. Our group needed proper communication between frontend and backend members to ensure that the displayed data matched the database records correctly. I also learned how important planning and task delegation are in completing a project efficiently. Monitoring our workflow and checking each member's progress helped us avoid major issues during integration. Overall, this phase strengthened our teamwork and collaboration skills.

Frontend Developer (Judy Pearl Balictar)

In this phase, I learned how to retrieve data from backend APIs and display it dynamically on the webpage using JavaScript and DOM manipulation. Creating reusable cards and dashboard components improved my understanding of frontend development and responsive design. I also learned how fetch() works with GET requests and how JSON data is processed on the client side. Some challenges included fixing rendering errors and making the interface more organized, but these experiences improved my debugging skills. This activity increased my confidence in building interactive web interfaces.

Backend Developer (John Roan Ballester)

This phase improved my understanding of backend API development and how frontend systems depend on properly structured GET endpoints. I learned how to retrieve data from MongoDB and return it in JSON format for frontend rendering. Testing the API responses using Postman and Thunder Client also helped me identify and fix issues before integration. Working closely with the frontend developer taught me the importance of consistent schema structures and proper API responses. Overall, this phase strengthened my backend development and troubleshooting skills.

Database Manager (Michelle Diaz)

During this phase, I learned how stored data in MongoDB can be retrieved and displayed dynamically on the frontend dashboard. I became more familiar with checking database records and ensuring that the schema fields matched the displayed information on the webpage. This phase also helped me understand how databases interact with APIs and frontend rendering processes. Monitoring the inserted records and verifying API outputs improved my attention to detail. Overall, the experience improved my understanding of database management in full-stack development.

GitHub Manager (Alyssa Jenille Reantaso)

This phase taught me the importance of version control and collaboration in software development projects. Although I experienced internet connection problems during the activity, I still learned how important proper commit organization and repository management are for group projects. Coordinating with my teammates helped ensure that project updates were still committed and synchronized properly. I also realized how teamwork and communication help overcome technical difficulties during development. This experience taught me the value of responsibility and cooperation in collaborative programming tasks.

Documentation Officer (Ronel Garcia)

This phase improved my skills in documenting workflows, organizing screenshots, and explaining technical processes clearly in the report. I learned how frontend rendering works and how data flows from the backend database to the user interface. Preparing annotations and descriptions for API responses, dashboard screenshots, and JavaScript snippets also improved my technical writing abilities. Observing the testing process helped me understand common issues encountered during frontend-backend integration. Overall, this activity strengthened both my documentation and debugging skills.